

Notebook

February 8, 2026

1 Атака на ошибки при работе RSA CRT (КТО)

Поскольку у RSA большое энергопотребление и он требует большой вычислительной мощности, на слабых устройствах часто используется вариант расшифрования на основе Китайской Теоремы об Остатках. Предварительно вычисляются значения $dP = e^{-1} \bmod (p-1)$, $dQ = e^{-1} \bmod (q-1)$ и $qInv = q^{-1} \bmod p$. После этого алгоритм расшифрования работает следующим образом:

$$M_p = C^{dP} \bmod p$$

$$M_q = C^{dQ} \bmod q$$

$$h = qInv \cdot (M_p - M_q) \bmod p$$

$$M = M_q + h \cdot q$$

$M_p = M \bmod p$ и $M_q = M \bmod q$

Алгоритм отлично работает, сильно уменьшая вычислительную сложность RSA, но у него есть один недостаток. Если внедрить ошибку(фолт) при вычислении одного из остатков M , то получится M' , для которого:

$$M'_p = M_p$$

$$M'_q \neq M_q$$

Раз $M'_p = M_p$, то $M - M' = kp$, для некоторого $k \in \mathbb{Z}$, и $GCD(M - M', N) = p$, что позволяет факторизовать N и вычислить d .

Получите с сервера M , M' , вычислите d и отправьте на сервер, чтобы получить флаг. Удачи!

```
[ ]: import socket
import re
from Crypto.Util.number import bytes_to_long, long_to_bytes, inverse, GCD
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""
        self.s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone', 1340))
        if show:
            print (self.recv_until().decode())
```

```

def recv_until(self, symb=b'\n>'):
    """Получаем сообщения с сервера, по дефолту до знака_\n>
    ↪приглашения"""
    data=b''
    while True:
        data+=self.s.recv(1)
        if data[-len(symb):]==symb:
            break
    return data

def get_public_key(self, show=True):
    """Получаем открытый ключ с сервера"""
    self.s.sendall('public\n'.encode())
    response=self.recv_until().decode()
    if show:
        print (response)
    e=int(re.search('(?<=e: )\d+', response).group(0))
    N=int(re.search('(?<=N: )\d+', response).group(0))
    self.num_len=len(long_to_bytes(N))
    return (e,N)

def decryptBytes(self, m, show=True):
    """Получить открытый текст от выбранного шифротекста в байтах_\n>
    ↪с сервера"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_\n>
    ↪first')
    return
    if len(m)>num_len:
        print ("The message is too long")
        return
    if len(m)<num_len:
        m=bytes((num_len-len(m))*[0x0])+m
    hex_m=m.hex().encode()
    self.s.sendall(b'decrypt '+hex_m+b'\n')
    response=self.recv_until().decode()
    if show:
        print (response)
    if response.find('flag')!=-1:
        print('You tried to submit \'flag\'')
        return None
    signature_hex=re.search('(?<=Signature: )[0-9a-f]+', response).
    ↪group(0)
    signature_bytes=bytes.fromhex(signature_hex)
    return bytes_to_long(signature_bytes)

```

```

def decryptNumber(self,m,show=True):
    """Получить открытый текст для выбранного закрытого текста в
↪ виде числа с сервера"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪ first')
    return
    return self.decryptBytes(long_to_bytes(m,num_len),show)

def faultyDecryptBytes(self,m,show=True):
    """Получить открытый текст с ошибкой в алгоритме для
↪ выбранного закрытого текста в байтах с сервера"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪ first')
    return
    if len(m)>num_len:
        print ("The message is too long")
        return
    if len(m)<num_len:
        m=bytes((num_len-len(m))*[0x0])+m
        hex_m=m.hex().encode()
        self.s.sendall(b'faulty_decrypt '+hex_m+b'\n')
        response=self.recv_until().decode()
        if show:
            print (response)
        if response.find('flag')!=-1:
            print('You tried to submit \'flag\'')
            return None
        signature_hex=re.search('(?!Signature: )[0-9a-f]+',response).
↪ group(0)
        signature_bytes=bytes.fromhex(signature_hex)
        return bytes_to_long(signature_bytes)

def faultyDecryptNumber(self,m,show=True):
    """Получить открытый текст с ошибкой в алгоритме для
↪ выбранного закрытого текста в числовом представлении с сервера"""
    try:
        num_len=self.num_len

```

```

        except AttributeError:
            print ('You need to get the public key from the server_
↪first')
            return
        return self.faultyDecryptBytes(long_to_bytes(m,num_len),show)

def checkDNumber(self,c,show=True):
    """Проверить, является ли это число d"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪first')
        return
    signature_bytes=long_to_bytes(c,num_len)
    self.checkDBytes(signature_bytes,show)

def checkDBytes(self,c,show=True):
    """Проверить, является ли эта последовательность байтов d"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪first')
        return
    if len(c)>num_len:
        print ("The message is too long")
        return

    hex_c=c.hex().encode()
    self.s.sendall(b'flag '+hex_c+b'\n',)
    response=self.recv_until(b'\n').decode()

    if show:
        print (response)

    if response.find('Wrong')!=-1:
        print('Wrong signature')
        x=self.recv_until()
        if show:
            print (x)
        return
    flag=re.search('CRYPTOTRAINING\{.*\}',response).group(0)
    print ('FLAG: ',flag)

def __del__(self):
    self.s.close()

```

```
[ ]: vs=VulnServerClient()  
    (e,N)=vs.get_public_key()
```

```
[ ]:
```